ENS de Lyon
Computer Science department

CGDI Project

Heat Kernel Signature for Mesh Classification using Halfedge CNN

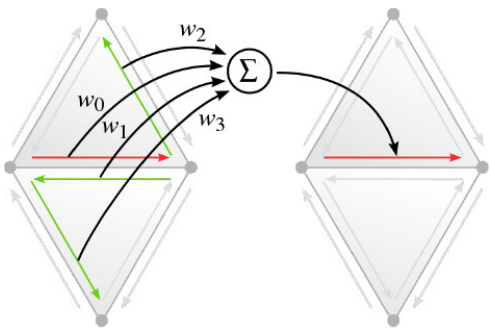


Figure 1: Mesh convolution

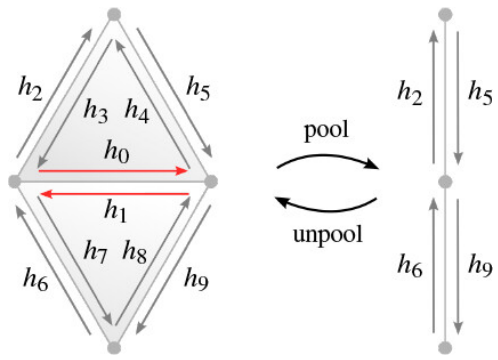


Figure 2: Mesh pooling

Name : Collas Estéban
Class : M1 computer science
Année universitaire : 2025–2026
Supervisors : V. Nivoliers / D. Coeurjolly

Done in Lyon, the March 27, 2026

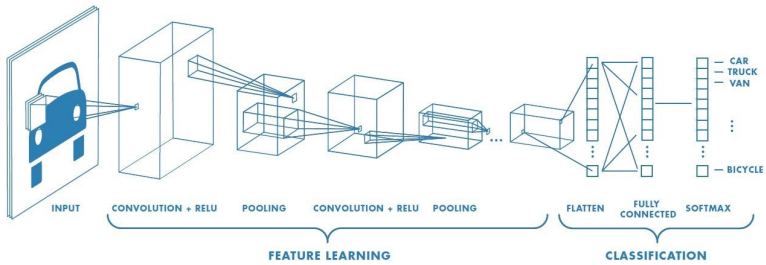


Figure 3: Typical architecture of a 2D Convolutional Neural Network.

1 Introduction :

1.1 Deep Learning on Meshes: MeshCNN and HalfedgeCNN

Convolutional Neural Networks (CNNs) are classical for image-based tasks, such as classification and segmentation, due to their ability to learn hierarchical features (2). As illustrated in Figure 3, a standard CNN architecture relies on two fundamental operations:

1. **Convolutions:** Extracting local features by sliding a kernel over a regular grid of pixels.
2. **Pooling:** Reducing the spatial resolution to gain a more global and abstract representation of the input.

While highly effective on regular grids (images), these operations do not directly translate to 3D meshes. To bridge this gap, (1) introduced **MeshCNN**, a framework that defines convolutions and pooling directly on the edges of a manifold mesh. Building upon this, **HalfedgeCNN** (3) further refines the approach by operating on *halfedges*. This choice provides increased flexibility and better handles the orientation of the surface, as each halfedge is uniquely associated with a single face.

Convolutions on Halfedges In the HalfedgeCNN framework, the convolution operation is defined over a local neighborhood. For a given halfedge, the neighborhood consists of adjacent halfedges (those sharing a vertex or belonging to the same face). Unlike 2D pixels, the number of neighbors can vary. In this project, I used the default neighborhood parameters as defined in the original literature (3). See Figure 13 for an illustration.

Mesh Pooling Pooling on a mesh requires a reduction in the number of primitives (eg. vertices, halfedges) while preserving the overall topology. In HalfedgeCNN, this is achieved by "flattening" or collapsing certain triangles. However, to ensure that the resulting structure remains a valid manifold mesh, the simplification must satisfy the **Link Condition** (3). This topological constraint prevents the creation of non-manifold vertices or edges during the collapse, ensuring the geometric integrity of the model throughout the layers. During pooling, we average all the vectors on the pooled halfedge (instead of taking the max value, for example). See Figure 14 for an illustration.

Input Features A critical design choice in mesh-based learning is the selection of input features attached to each halfedge. To ensure the model is invariant to rigid transformations (rotation and translation), the features must be **intrinsic**. The following configurations are explored in (3):

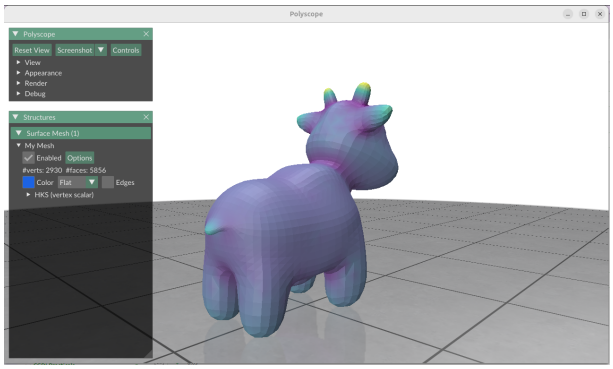


Figure 4: HKS of a cow

- **Symmetric Data:** Includes dihedral angles, symmetric opposite angles, and symmetric ratios. This setup is similar to the original MeshCNN and focuses on local curvature.
- **Asymmetric Data:** Incorporates the specific orientation of halfedges by using non-symmetric ratios and opposite angles. This allows us to use the asymmetry of the HalfedgeCNN structure.
- **Minimal Data:** A reduced set consisting of dihedral angles and normalized edge lengths, providing a baseline for geometric complexity.

The detailed implementation of these feature extractors can be found in the `models/layers/half_edge_mesh_prepare` module on the github.

1.2 Heat Kernel Signature

The Heat Kernel Signature (HKS) is a point descriptor based on the properties of the heat diffusion equation on a manifold. Unlike the geometric features mentioned above (angles, ratios), the HKS captures the intrinsic structure of the mesh, making it invariant to isometric transformations (i.e., changes in the pose of the object that do not stretch its surface) (4). See Figure 4 for the HKS of a cow.

The diffusion of heat on a surface is governed by the heat equation:

$$\frac{\partial u}{\partial t} = \Delta u \tag{1}$$

where Δ is the Laplace-Beltrami operator (implemented as in the TP). The HKS at a point x and time t represents the amount of heat remaining at x after a period t, starting from a unit point source at $t = 0$. This can be computed using the spectral decomposition of the Laplace-Beltrami operator:

$$HKS(x, t) = \sum_{i=0}^{\infty} e^{-\lambda_i t} \phi_i(x)^2 \tag{2}$$

where λ_i and ϕ_i are the i-th eigenvalues and eigenfunctions of the operator, respectively. In practice, on a discrete mesh, this sum is truncated to the first k eigenvalues (typically $k \in [50, 100]$) computed from the cotangent stiffness matrix W and the mass matrix M.

The HKS offers two major advantages for mesh classification:

- **Multi-scale nature:** For small values of t, the HKS captures local curvature (detecting tips, corners, and small details). For larger t, it reflects the global structure of the shape (distinguishing between massive bodies and elongated limbs).

- **Invariance:** Since it depends only on the eigenvalues of the Laplace-Beltrami operator, it remains identical even if the mesh is rotated or deformed into a different pose.

In the context of our HalfedgeCNN, the HKS is natively a vertex-based scalar. To integrate it into the model's input vector, we perform a linear interpolation for each halfedge. For a halfedge h_{ij} connecting vertex v_i to v_j, the value attached to the edge is defined as the maximum of the HKS values at its endpoints:

$$HKS(h_{ij}, t) = max(HKS(v_i, t), HKS(v_j, t)) \tag{3}$$

By concatenating the HKS values at different time scales t with the classical geometric features (dihedral angles, lengths), the model gains access to both local differential geometry and global topological information.

I also tried to use the difference betwenn the two HKS values, but it did not work as well as the max.

2 Contributions :

The goal of my project was to understand the implications of using HKS as input for the HalfedgeCNN model.

In order to see this, I adapted the code from (3) to use HKS and save data about what each channel in the model was doing with HKS.

I had 4 phases in my project:

Understand the code from HalfedgeCNN I spent time reading the article (3) and the code from their github, to understand how their model was really working. I also created the following colab to execute their code.

Understanding and implementation of the HKS I learned about HKS and implemented it, based on what we did during the TP. This led to the file `manual_calcuations/calculate_hks.ipynb` (github), containing an implementation of the HKS for a .obj file.

Insertion of HKS in HalfedgeCNN The biggest part of the project was about implementing HKS into HalfedgeCNN. This led to all the other files described in the README.md (github).

Experiment with the HKS model Once the model was able to use HKS, I spent time training it on a second google colab and plotting the "updated HKS values" (see Annex 1) on test data, to try to understand what the model was doing with the HKS.

3 Choices

3.1 One value of t for the HKS

Normally, people use multiple values of t for the HKS. Personally, I used only one ($t = 0.01$).

I wanted to see whether the model was able to classify the meshes with only one value for the HKS. I trained a model which attained $\sim 90\%$ accuracy using only this one value of t and ~ 100 epochs (and no other data as input). I plan to do a longer training during the weekend to see whether I can surpass this accuracy.

Also, it was easier to "see through the eyes of the model" with only one value for t, because I could just plot the HKS at a given time on the pooled meshes, as you can see in the github.

It would be easy to adapt the code to use multiple values.

3.2 storage of some "HKS values"

In order to "see through the eyes of the model", when testing a model, my code stores the HKS values of a given chosen channel before each pooling operation, as well as the structure of the pooled mesh. Using `util/mesh_viewer_polyscope`, one can then see the evolution of the HKS values as the model applies convolutions and poolings on the mesh.

The choice of storing the HKS values noly before the pooling operations is quite arbitrary, but I assumed it was enough to see what was going on, without storing too much data. It would be possible to also store the HKS values before convolutions.

4 Results

To reproduce my results, see the instructions in the github

4.1 Efficiency of the HKS

The original article acheived an accuracy of $\sim 99\%$ with 200 epochs, using Asymmetric Data. Using only HKS with only 1 value and 100 epochs, I achieved an accuracy of $\sim 90\%$. I plan to experiment a bit more to get clearer results (the training is pretty long, even on google colab), but those results are very promising. The HKS seems indeed to caracterize most of what the model wants to know about the mesh. See Annex 2 for some output of the training loop on colab.

4.2 Evolution of the HKS for an untrained model

The pooling operation is an average. Hence, when the model is untrained (at the beginning), the HKS seem to be averaged on the mesh (see Figure 5-8).

I saw this using `util/mesh_viewer_polyscope`.

5 Discussion

As said above, the HKS seems indeed to caracterize most of what the model wants to know about the mesh. Using multiple values for t instead of only one, I think I could reach an even better accuracy. This needs more experimentation.

I also found it interesting to be able to visualise to evolution of the HKS values during the execution of the model. I believe it could help to have an intuition about which information is in the HKS, and which information is lacking, if we cannot reach an accuracy as high as with the Asymmetric Data using only HKS.

I spent a good deal of time on the project on "details", like modifying the code in order to make it easier to use from the command line, etc.

References

[1] Rana Hanocka, Amir Hertz, Noa Fish, Raja Giryes, Shachar Fleishman, and Daniel Cohen-Or. Meshcnn: a network with an edge. *ACM Transactions on Graphics (ToG)*, 38(4):1–12, 2019.

[2] Yann LeCun and Yoshua Bengio. Convolutional networks for images, speech, and time series. *The handbook of brain theory and neural networks*, 1998.

[3] I. Ludwig, D. Tyson, and M. Campen. Halfedgecnn for native and flexible deep learning on triangle meshes. *Computer Graphics Forum*, 42(5), 2023.

[4] Jian Sun, Maks Ovsjanikov, and Leonidas Guibas. A concise and provably informative multi-scale signature based on heat diffusion. In *Computer graphics forum*, volume 28, pages 1383–1392. Wiley Online Library, 2009.

6 Annexes

6.1 Annex 1

See Figure 5-8 for a visualisation of the operations done by a weak model (accuracy $\sim 12\%$) and Figure 9-12 for a strong model (accuracy $\sim 90\%$) on the same mesh. What is happening is not so clear, but there seem to be more structure in the HKS values from the strong model. For example, I think that Figure 6 could correspond to random operations between the HKS values.

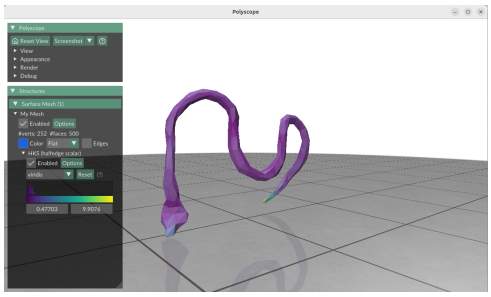


Figure 5: Snake - initial HKS values

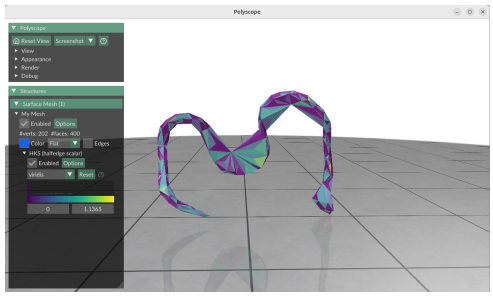


Figure 6: Snake - before second pooling of the weak model (first channel)

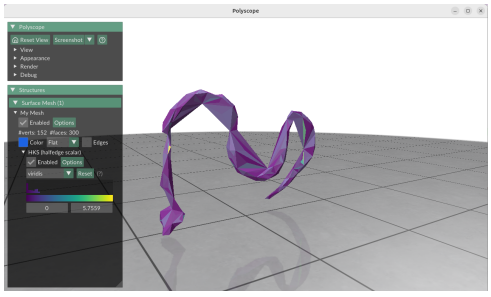


Figure 7: Snake - before third pooling of the weak model (first channel)

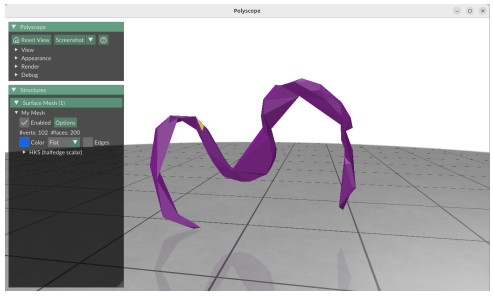


Figure 8: Snake - before fourth pooling of the weak model (first channel)

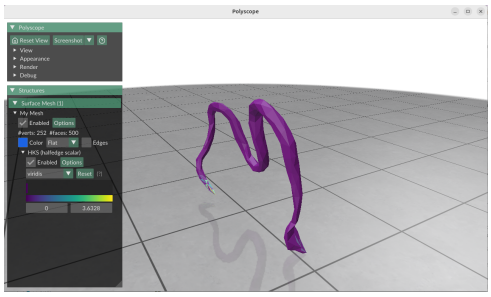


Figure 9: Snake - before first pooling of the strong model (first channel)

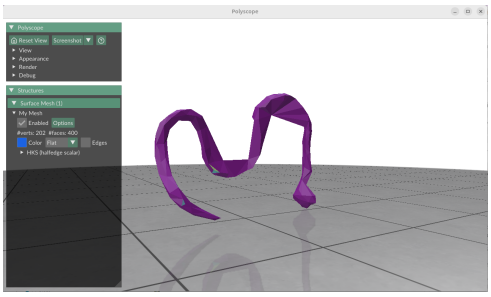


Figure 10: Snake - before second pooling of the strong model (first channel)

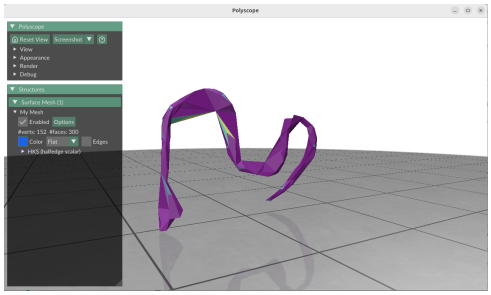


Figure 11: Snake - before third pooling of the strong model (first channel)

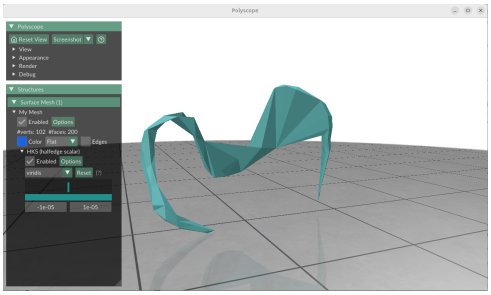
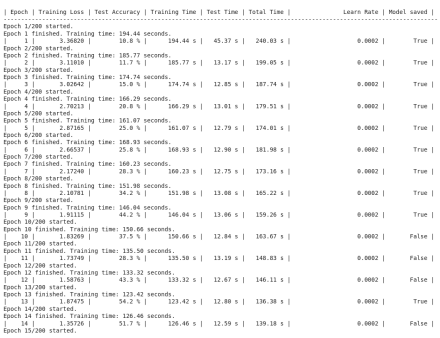


Figure 12: Snake - before fourth pooling of the strong model (first channel)

6.2 Annex 2

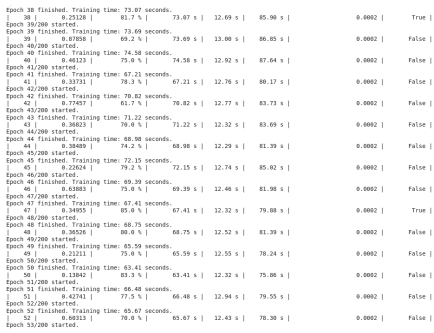
The training of the model was made using Adam. This means we only use a subset of the data at each step of the gradient descent, explaining the fact that sometimes, the training loss increases. Also, this explains why the training is faster and faster. Indeed, we can store pre-computed information about the meshes (also, maybe colab allow us more ressources as we execute many operations).

I took screenshots of the colab during training. You can see them in the Figures 13-14. The training was pretty irregular. I think this is because of the Adam optimizer. As mentioned in the colab, it is not good practice to choose the model using the test set as I do here (following the work in (3)). It would be better to use a validation set.



```
...  | Epoch | Training Loss | Test Accuracy | Training Time | Test Time | Total Time |            Learn Rate | Model saved |
-----------------------------------------------------------------------------------------------------------------------------
Epoch 1/200 started.
Epoch 1 finished. Training time: 194.44 seconds.
|     1 |       3.36820 |          10.8 % |     194.44 s |   45.37 s |   240.03 s |                0.0002 |        True |
Epoch 2/200 started.
Epoch 2 finished. Training time: 185.77 seconds.
|     2 |       3.11810 |          11.7 % |     185.77 s |   13.17 s |   199.05 s |                0.0002 |        True |
Epoch 3/200 started.
Epoch 3 finished. Training time: 174.74 seconds.
|     3 |       3.02642 |          15.8 % |     174.74 s |   12.85 s |   187.74 s |                0.0002 |        True |
Epoch 4/200 started.
Epoch 4 finished. Training time: 166.29 seconds.
|     4 |       2.70213 |          20.8 % |     166.29 s |   13.01 s |   179.51 s |                0.0002 |        True |
Epoch 5/200 started.
Epoch 5 finished. Training time: 161.07 seconds.
|     5 |       2.87165 |          25.0 % |     161.07 s |   12.79 s |   174.01 s |                0.0002 |        True |
Epoch 6/200 started.
Epoch 6 finished. Training time: 168.93 seconds.
|     6 |       2.66537 |          25.8 % |     168.93 s |   12.90 s |   181.98 s |                0.0002 |        True |
Epoch 7/200 started.
Epoch 7 finished. Training time: 160.23 seconds.
|     7 |       2.17240 |          28.3 % |     160.23 s |   12.75 s |   173.16 s |                0.0002 |        True |
Epoch 8/200 started.
Epoch 8 finished. Training time: 151.98 seconds.
|     8 |       2.10791 |          34.2 % |     151.98 s |   13.08 s |   165.22 s |                0.0002 |        True |
Epoch 9/200 started.
Epoch 9 finished. Training time: 146.04 seconds.
|     9 |       1.91115 |          44.2 % |     146.04 s |   13.06 s |   159.26 s |                0.0002 |        True |
Epoch 10/200 started.
Epoch 10 finished. Training time: 150.66 seconds.
|    10 |       1.83269 |          37.5 % |     150.66 s |   12.84 s |   163.67 s |                0.0002 |       False |
Epoch 11/200 started.
Epoch 11 finished. Training time: 135.50 seconds.
|    11 |       1.73749 |          28.3 % |     135.50 s |   13.19 s |   148.83 s |                0.0002 |       False |
Epoch 12/200 started.
Epoch 12 finished. Training time: 133.32 seconds.
|    12 |       1.50763 |          43.3 % |     133.32 s |   12.67 s |   146.11 s |                0.0002 |       False |
Epoch 13/200 started.
Epoch 13 finished. Training time: 123.42 seconds.
|    13 |       1.07475 |          54.2 % |     123.42 s |   12.80 s |   136.38 s |                0.0002 |        True |
Epoch 14/200 started.
Epoch 14 finished. Training time: 126.46 seconds.
|    14 |       1.35726 |          51.7 % |     126.46 s |   12.59 s |   139.18 s |                0.0002 |       False |
Epoch 15/200 started.
```

Figure 13: Training HKS (beginning)



```
Epoch 38 finished. Training time: 73.07 seconds.
|    38 |       0.25128 |          81.7 % |      73.07 s |   12.69 s |   85.90 s |                0.0002 |        True |
Epoch 39/200 started.
Epoch 39 finished. Training time: 73.69 seconds.
|    39 |       0.87858 |          69.2 % |      73.69 s |   13.00 s |   86.85 s |                0.0002 |       False |
Epoch 40/200 started.
Epoch 40 finished. Training time: 74.58 seconds.
|    40 |       0.46123 |          75.0 % |      74.58 s |   12.92 s |   87.64 s |                0.0002 |       False |
Epoch 41/200 started.
Epoch 41 finished. Training time: 67.21 seconds.
|    41 |       0.33731 |          78.3 % |      67.21 s |   12.76 s |   80.17 s |                0.0002 |       False |
Epoch 42/200 started.
Epoch 42 finished. Training time: 70.82 seconds.
|    42 |       0.77457 |          61.7 % |      70.82 s |   12.77 s |   83.73 s |                0.0002 |       False |
Epoch 43/200 started.
Epoch 43 finished. Training time: 71.22 seconds.
|    43 |       0.36823 |          70.0 % |      71.22 s |   12.32 s |   83.69 s |                0.0002 |       False |
Epoch 44/200 started.
Epoch 44 finished. Training time: 68.98 seconds.
|    44 |       0.38489 |          74.2 % |      68.98 s |   12.29 s |   81.39 s |                0.0002 |       False |
Epoch 45/200 started.
Epoch 45 finished. Training time: 72.15 seconds.
|    45 |       0.22624 |          79.2 % |      72.15 s |   12.74 s |   85.02 s |                0.0002 |       False |
Epoch 46/200 started.
Epoch 46 finished. Training time: 69.39 seconds.
|    46 |       0.63883 |          75.0 % |      69.39 s |   12.46 s |   81.98 s |                0.0002 |       False |
Epoch 47/200 started.
Epoch 47 finished. Training time: 67.41 seconds.
|    47 |       0.34955 |          85.0 % |      67.41 s |   12.32 s |   79.88 s |                0.0002 |        True |
Epoch 48/200 started.
Epoch 48 finished. Training time: 68.75 seconds.
|    48 |       0.30528 |          80.0 % |      68.75 s |   12.52 s |   81.39 s |                0.0002 |       False |
Epoch 49/200 started.
Epoch 49 finished. Training time: 65.59 seconds.
|    49 |       0.21211 |          75.0 % |      65.59 s |   12.55 s |   78.24 s |                0.0002 |       False |
Epoch 50/200 started.
Epoch 50 finished. Training time: 63.41 seconds.
|    50 |       0.13842 |          83.3 % |      63.41 s |   12.32 s |   75.86 s |                0.0002 |       False |
Epoch 51/200 started.
Epoch 51 finished. Training time: 66.40 seconds.
|    51 |       0.42741 |          77.5 % |      66.40 s |   12.94 s |   79.55 s |                0.0002 |       False |
Epoch 52/200 started.
Epoch 52 finished. Training time: 65.67 seconds.
|    52 |       0.66313 |          70.0 % |      65.67 s |   12.43 s |   78.30 s |                0.0002 |       False |
Epoch 53/200 started.
```

Figure 14: Training HKS (later)